NUMERICAL ROOT FINDER

APPDAET Progress Report 1 · Group 6

Aaron Natividad, Kyra Lim, Reine Co, Akima Fernandez,
Nathan Garcia

PROJECT PURPOSE

- Approximate one real root of a cubic polynomial.
- Reduce repetitive calculations and manual errors.
- Show how the Bisection Method narrows the interval.
- Planned Python desktop app with a table and graph.

CORE REQUIREMENTS

01 3 preset equations + custom coefficients

02 Adjustable bounds, tolerance and iteration limit

03 Student-developed Bisection algorithm

04 Iteration table and graph with root marker

05 Input validation; root, residual, count and stop reason

Inputs, Processing & Outputs

$f(x) = ax^3 + bx^2 + cx + d$, where $a \neq 0$

Inputs

Coefficients a, b, c, d
Bounds L and U
Positive tolerance
Positive integer limit

→

Processing

Validate inputs
Check endpoint roots
Bisect the bracket
Record each iteration

→

Outputs

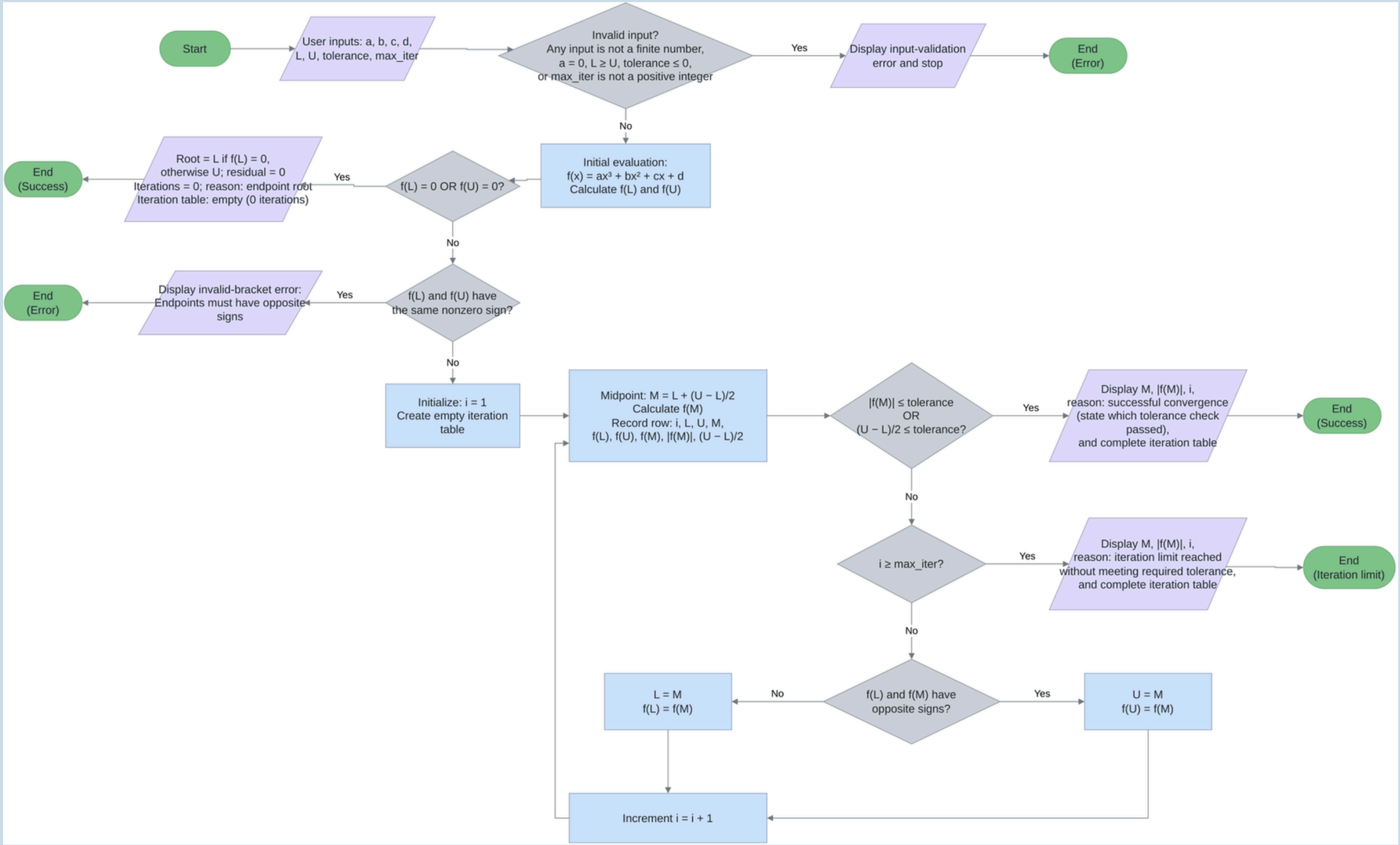
Root and residual
Count and stop reason
Iteration table
Polynomial graph

After endpoint checks, require $f(L)f(U) < 0$.

SCOPE & LIMITATIONS

- Cubic equations: $f(x) = ax^3 + bx^2 + cx + d$, $a \neq 0$.
- One bracketed real root per run.
- Accept an endpoint that is already a root.
- Otherwise require $L < U$ and $f(L)f(U) < 0$.
- Accuracy depends on tolerance and the iteration limit.
- Invalid bracket does not mean no real root exists.
- Outside current scope: all roots and Newton-Raphson.

Algorithm Flowchart



Proposed Functions & Program Structure

1. get_user_input() → validate_inputs()

↓

2. check_interval() — endpoint roots and signs

↓

3. bisection_method()

Calls evaluate_polynomial(); records iteration history

↓

4. display_results() + plot_polynomial()

Returned data

Root estimate and residual
Iteration count and status
Complete iteration history

Proposed modules

main.py / gui.py
bisection.py / polynomial.py
validation.py / presets.py
plotting.py / tests/

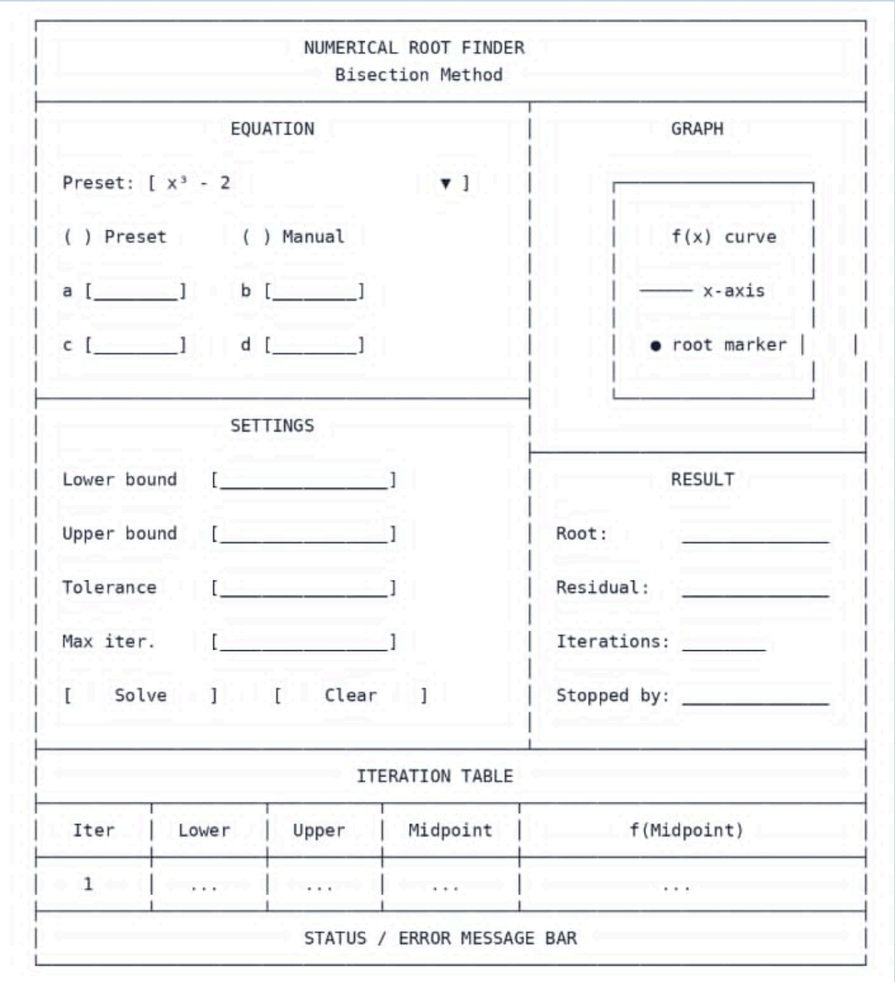
DEVELOPMENT ENVIRONMENT

PROPOSED TOOLS
Python + VS Code for development
Tkinter/ttk for the desktop interface
Matplotlib for graphing; NumPy if needed for graph samples
unittest for tests; Git + GitHub for collaboration

SUPPORTING RESOURCES
Course materials on the Bisection Method
Python/Tkinter and Matplotlib documentation

The root-finding algorithm will use our own Python functions.

INITIAL GUI WIREFRAME



INPUTS
Equation and calculation settings

RESULTS
Root, residual, iterations and stopping reason

VISUAL OUTPUTS
Graph and iteration table

FEEDBACK
Status and error messages

GITHUB REPOSITORY & WORKFLOW

PUBLIC REPOSITORY
https://github.com/drake12324/Numerical-Root-Finder

VERIFIED SETUP
README.md and .gitignore present
Folders: docs/, gui/ and tests/
Initial commits include documentation, GUI and testing folders

PROPOSED WORKFLOW
Pull → edit → test → commit → push
Collaborator and instructor access: not independently verified.

Development Timeline

Stage	Planned work	Expected output
1 · Planning	Scope, flowchart, wireframe, assignments, GitHub	Progress Report 1
2 · Core program	Polynomial evaluation, validation, Bisection	Working algorithm and progress report
3 · GUI integration	Interface, iteration table and graph	Initial desktop application
4 · Testing	Known roots, invalid inputs and error fixes	Test results and progress report
5 · Final submission	README, screenshots, files and presentation	Complete final project

Planned sequence; dates follow the professor’s announced deadlines.

Member Responsibilities

Member	Assigned area	Expected output
Aaron Natividad	Parts A and E	Requirements, task matrix and timeline
Kyra Michiko Lim	Algorithm design (B1–B2)	Bisection explanation and flowchart
Reine Nathaniel Co	Decomposition (B3–B5)	Components, functions and interactions
Akima Lou Fernandez	Technical preparation (C)	Tools, formulas, GUI and file structure
Nathan Drake Garcia	GitHub initialization (D)	Repository, README and commit evidence

All members contribute to development, testing, review and presentation.

THANK YOU

Presented by Group 6